TradeSmart AI

An AI-powered vehicle trade-in valuation assistant

Contents

Contents 2
1. The AI Solution and Its Relevance to the Theme 3
2. Problem Definition 4
  2.1 What is the problem? 4
  2.2 Relevance to the theme 4
  2.3 Benefits of solving it with AI 4
3. Main Objective and How AI Is Applied 5
  3.1 Main objective 5
  3.2 How AI is applied 5
4. Business Objectives 6
  4.1 Business background 6
  4.2 Business objectives 6
  4.3 Business success criteria 6
5. Requirements, Constraints, Risks and Tools 8
  5.1 Functional requirements 8
  5.2 Non-functional requirements 8
  5.3 Constraints 9
  5.4 Risks and mitigations 9
  5.5 Initial assessment of tools and techniques 10
6. Machine Learning Approach 11
7. Data 12
  7.1 Source 12
  7.2 Forms of data used 12
  7.3 Data quality: errors found and corrected 13
  7.4 Database 13
8. Model Evaluation and Time Series Analysis 15
  8.1 How accuracy is evaluated 15
  8.2 Results 15
  8.3 Time series analysis: how a car's value changes over time 17
9. Solution Techniques and Improving Accuracy 20
  9.1 Techniques used 20
  9.2 How the model's accuracy is improved 20
10. Natural Language Processing 22
11. Deep Learning 24
12. Chatbot 25
13. Practical Solution 26
References 27

1. The AI Solution and Its Relevance to the Theme

TradeSmart AI is a conversational assistant built for South African used-vehicle dealerships, an application of AI in the automotive retail industry, one of the sectors named in the project theme "an AI Solution for Industries". The dealership deploys it on its website or on a showroom tablet, and its sales consultants use the result as the starting point for their appraisal. The assistant collects the vehicle's make, model, year and mileage, and an XGBoost machine-learning model trained on 16 453 real Autotrader South Africa listings (model years 2018-2023) predicts its market value range. Natural language processing (NLP) interprets the customer's own description of accidents, dents, windscreen damage, respray work and modifications, and records each issue for the consultant to check at inspection. Every valuation is saved in the dealership's database. In line with the Fourth Industrial Revolution, the solution turns a slow, manual and inconsistent process into a fast, repeatable, data-driven one using locally relevant data and open-source tools. A deep-learning model was also built and evaluated against XGBoost (Section 11).

2. Problem Definition

2.1 What is the problem?

Used-vehicle dealerships in South Africa value trade-in vehicles slowly and inconsistently. A first-pass appraisal depends on a sales consultant's experience and a manual search of guide prices and online listings, so two consultants can value the same vehicle differently. A wrong valuation costs the dealership either way: overpaying loses margin, and underpaying loses the deal to a private sale or a competitor. Customers also wait for a figure, which delays negotiation, and what they say about the car's condition (accidents, dents, modifications) is captured informally and often lost before inspection. Because appraisals are not recorded in a structured way, the dealership cannot learn from past enquiries. The problem is the lack of a fast, consistent, data-driven way to produce a first value estimate and to record a vehicle's reported condition.

2.2 Relevance to the theme

This is an operational problem in the automotive retail industry that machine learning and conversational AI can address directly, which matches the theme 'an AI Solution for Industries'.

2.3 Benefits of solving it with AI

The assistant gives customers and consultants a consistent, data-based starting valuation within seconds, saves consultant time, reduces mispriced trade-ins and builds a valuation dataset the dealership owns. As this is a business problem it does not involve a local municipality, although faster, fairer valuations help local dealerships win more trade-ins.

3. Main Objective and How AI Is Applied

3.1 Main objective

To develop a conversational AI assistant for dealerships that collects a vehicle's details, records its reported condition using NLP, and produces a data-driven market value range for the consultant.

3.2 How AI is applied

- **Chatbot** to collect the vehicle's details and the customer's answers (Section 12).
- **XGBoost regression** to predict the vehicle's value from its make, model, year and mileage (Sections 6 and 8).
- **NLP** (TF-IDF and Logistic Regression) to turn free-text condition descriptions into categorised findings (Section 10).
- **Deep learning:** a neural network (MLP) trained on the same data and compared with XGBoost (Section 11).
- **Time series analysis** of how vehicle values change with age (Section 8.3).
- **Hyperparameter tuning** to improve the model's accuracy (Section 9.2).

4. Business Objectives

4.1 Business background

The solution is designed for a medium-sized used-vehicle dealership, referred to in this document as AutoValue Motors, a fictitious dealership in Vanderbijlpark. Like most dealerships, it accepts customers' current vehicles as trade-ins when they buy another vehicle. Trade-in values are decided manually by individual sales consultants, who compare guide prices and similar cars advertised online. This is slow and inconsistent: two consultants can value the same car differently, and a wrong valuation either costs the dealership margin (overpaying) or loses the deal (underpaying). Customers expect quick answers, and the dealership keeps no structured record of its valuations.

4.2 Business objectives

1. Reduce the time and staff effort needed for a first-pass appraisal: customers get a first estimate in one chat conversation, at any time, without logging in.
2. Standardise valuations so that similar vehicles get similar estimates, and the same details always give the same estimate.
3. Give consultants an accurate, data-driven starting value range to support negotiation.
4. Record the vehicle's reported condition with each valuation, as structured, categorised findings for inspection.
5. Build a valuation dataset, owned by the dealership, for future model improvement.
6. Protect customers' personal information in line with the Protection of Personal Information Act (POPIA).

4.3 Business success criteria

The first six criteria were measured on the finished solution. The last two can only be measured once the dealership is using it, so they are targets for after deployment.

Criterion	Measure	Target	Result
Estimate accuracy	Unseen test vehicles whose estimate is within ±15% of the actual listing	≥ 80%	Achieved: 87.8% (75.0% within ±10%)

Criterion	Measure	Target	Result
	price (average of 5 random 80/20 splits)		
Overall accuracy	R^2 on unseen test vehicles (average of 5 random 80/20 splits)	≥ 0.90 and better than the depreciation baseline	Achieved: 0.911 on average (range 0.859-0.952); baseline 0.902
Response speed	Time to show the estimate after the last answer	< 3 seconds	Achieved: 11 ms typical, 44 ms slowest (20 runs, excluding the database save)
NLP accuracy	Condition classifier on labelled held-out statements	≥ 85%	Achieved: 96% on 26 statements
POPIA compliance	Contact details stored only with consent	100%	Achieved: enforced in the database and in the log file
Consistency	Same vehicle details give the same estimate	100%	Achieved: the trained model is deterministic
Appraisal time	First-pass appraisal time compared with the manual method	60% reduction	After deployment
Valuation complaints	Valuation complaints recorded by the dealership	30% reduction	After deployment

5. Requirements, Constraints, Risks and Tools

5.1 Functional requirements

- **FR1** A chat interface where the customer finds their vehicle by typing its make and model and choosing from matching vehicles.
- **FR2** The customer enters the vehicle's year and mileage; unreadable values are rejected with a clear message on the same step.
- **FR3** The customer can describe accidents, damage and modifications in free text, or skip the question.
- **FR4** NLP interprets the condition answer and stores each issue as a condition finding, flagging uncertain or contradictory ones.
- **FR5** The customer can optionally add a cash deposit towards another vehicle.
- **FR6** POPIA consent is required before phone and email are stored.
- **FR7** The system shows a value estimate with a low and high range and the model's measured accuracy, and flags vehicles whose year is outside 2018-2023.
- **FR8** Every session is saved in the database and appended to logs/customer_submissions.csv, which opens in Excel.

The estimate is a starting point, not an offer: a consultant confirms the final figure when the vehicle is inspected.

5.2 Non-functional requirements

- **Performance:** the estimate is shown in under 3 seconds after the last answer.
- **Usability:** easy to use without a login or any training; works in any modern web browser.
- **Privacy and security:** minimal personal data, consent-based storage of contact details, and no personal data in the source-code repository.
- **Reliability:** if the database is unavailable the customer still receives the estimate, and a locked log file never interrupts the chat.
- **Scalability:** able to grow from one branch to many; the database separates customers, vehicles, models and valuation sessions.

- **Maintainability:** the model can be retrained with a short sequence of documented scripts, and package versions are pinned.

5.3 Constraints

- **No real trade-in records:** the model is trained on public Autotrader asking prices, which are normally higher than trade-in offers.
- **Model years 2018-2023 only:** the data cannot reliably value older vehicles.
- **No condition information in the data:** so condition is recorded for inspection but does not change the price.
- **No price history:** the data is a single snapshot (April 2023), which limits the time series analysis (Section 8.3).
- **Limited time and budget:** an academic timeline and only free, open-source tools.
- **A consultant must make the final decision:** the estimate supports the appraisal but does not replace it.

5.4 Risks and mitigations

Risk	Impact	Mitigation
Asking prices are higher than trade-in prices, so the dealership could overpay	High	The output is a market value range; the consultant sets the offer after inspection; later calibration against real agreed prices (Section 9.2)
The estimate is treated as a firm offer	Medium	A disclaimer is shown with every range
Estimates for vehicles outside 2018-2023 are unreliable	High	Low-confidence warning in the chat; OutsideTrainingYears flag in the database
NLP misreads a vague answer	Low	Confidence score and NeedsReview flag stored; findings never change the price
Errors in the source data	High	Three cleaning errors were found and corrected before training (Section 7.3)
Personal information is leaked	High	POPIA consent tick box, minimal fields, customer log excluded from version control
The model becomes outdated as prices change	Medium	Documented retraining pipeline (Section 9.2)

Risk	Impact	Mitigation
A package update breaks the saved model	Medium	Exact package versions are pinned

5.5 Initial assessment of tools and techniques

Tool / technique	Used for	Why it was chosen
Python 3.12	All code	The language required by the module; strong data-science libraries
pandas, NumPy	Data loading, cleaning, feature engineering	Standard, well-documented tools for tabular data
scikit-learn	Linear Regression, Random Forest, preprocessing, cross-validation, tuning, TF-IDF + Logistic Regression	One consistent API for many models and for fair evaluation
XGBoost	Final valuation model	Best accuracy on tabular data in our comparison; fast; small model file
TensorFlow / Keras	Deep learning (MLP) model	Industry-standard deep learning library
Flask, Jinja2, htmx	Web chatbot	Lightweight Python web framework; the chat needs no hand-written JavaScript
Microsoft SQL Server	Database	Available at VUT; relational design with enforced relationships
matplotlib	Figures for the time series analysis	Standard Python plotting library
GitHub	Version control and project management	Required by the brief

6. Machine Learning Approach

Estimating a car's value is a **supervised regression** problem: the model learns from thousands of cars whose price is known and predicts a number (the price) for a car it has not seen. Each car is described by its make and model specifications (engine size, power, torque, fuel consumption, body type, drive type and so on), its age and its mileage.

Five approaches were trained on exactly the same data and compared on the same unseen test cars:

- **Depreciation baseline:** the rule of thumb that the AI must beat. It takes the price of the newest examples of a model and reduces it by a fixed percentage per year of age.
- **Linear Regression:** a simple model that assumes price changes in straight lines with each feature.
- **Random Forest:** many decision trees trained on random samples of the data, whose predictions are averaged (Breiman, 2001).
- **XGBoost:** gradient-boosted decision trees, where each new tree corrects the errors of the previous ones (Chen and Guestrin, 2016).
- **Deep learning (MLP):** a multi-layer neural network (Section 11).

Tree-based ensembles were expected to perform best because the value of a car depends on interactions between features: an extra 50 000 km matters far more on a sports car than on a bakkie, which a straight-line model cannot express. XGBoost was selected for the application because it had the lowest RMSE and highest R^2 (it makes the fewest large mistakes) and its saved model is small enough to load quickly; the Random Forest model file is about 100 MB.

A customer does not know their car's torque or CO2 rating, so the application uses the **typical specification of the exact model** they choose: the median of each numeric specification and the most common value of each category across all listings of that model. This is stored in the VEHICLE_MODEL table, summarised from the 16 453 listings in the VEHICLE table. Only the customer's own year and mileage are entered per valuation, which is how a human consultant would reason about a car.

7. Data

7.1 Source

The model is trained on **Autotrader South Africa** used-car listings. The data was collected by Adewoyin (2023), who scraped the website between 23 and 24 April 2023 and published the cleaned dataset and code under the MIT licence. It contains listings for model years 2018 to 2023. After our corrections (Section 7.3) it holds **16 453 listings** of **2 393 makes and models** from 53 brands, priced from R59 000 to R12 million (median R379 900).

7.2 Forms of data used

Form	Examples	Where it is used
Structured numerical	Price, mileage, year, engine capacity, power (kW), torque (Nm), fuel consumption, CO2	Model features and target (price)
Structured categorical	Brand, transmission, fuel type, body type, driven wheels, cylinder layout	Model features (one-hot encoded)
Text (free text)	Customer answers such as "small dent about the size of a golf ball on the door"	NLP condition findings
Labelled text	102 example customer answers, each labelled with one of 8 categories	Training the NLP classifier
Personal data	Customer name, phone, email (with consent)	CUSTOMER table

Example listings from the dataset (a subset of the 26 columns):

Make and model	Year	Mileage	Body	Power	Price
Toyota Hilux 2.4GD-6 Double Cab Raider	2023	5 500 km	Double cab	110 kW	R529 900
Volkswagen Polo Vivo Hatch 1.4 Trendline	2023	5 500 km	Hatchback	55 kW	R259 900
BMW X5 xDrive30d M Sport	2022	30 000 km	SUV	195 kW	R1 339 900

7.3 Data quality: errors found and corrected

While building the solution we found three errors that the published cleaning step had introduced, and we corrected them before any model was trained (tradesmart-ml/src/fix_dataset.py):

- **Split brand names (316 listings).** The brand was taken as the first word of the advert title, so "Land Rover Defender" became brand "Land", model "Rover Defender". The same happened to Alfa Romeo and Aston Martin.
- **Fuel consumption missing its leading "1" (405 listings restored, 179 removed).** A text-stripping step removed the characters 1, 0, k and m from both ends of each value, so 11.5 L/100 km became 0.5. No listing in the published data had a value between 10 and 19.9. We restored each value where the tank size and driving range confirmed it, and blanked those that could not be confirmed.
- **Impossible mileages (3 listings removed),** for example 990 000 km on a 2018 vehicle.

7.4 Database

The data and every valuation are stored in Microsoft SQL Server in six related tables (Figure 1). VEHICLE holds the full dataset: all 16 453 Autotrader listings, each linked to its make and model in VEHICLE_MODEL, which holds one row per make and model (2 393) with the typical specifications the model uses. CUSTOMER records who asked for a valuation; VALUATION_SESSION records the customer's car (which make and model, its year and mileage) and the predicted price; CONDITION_FINDING stores each condition issue the NLP model found; CONDITION_CATEGORY lists the eight categories it can assign. A check in SQL confirms that every VEHICLE_MODEL row's listing count matches the number of VEHICLE listings linked to it.

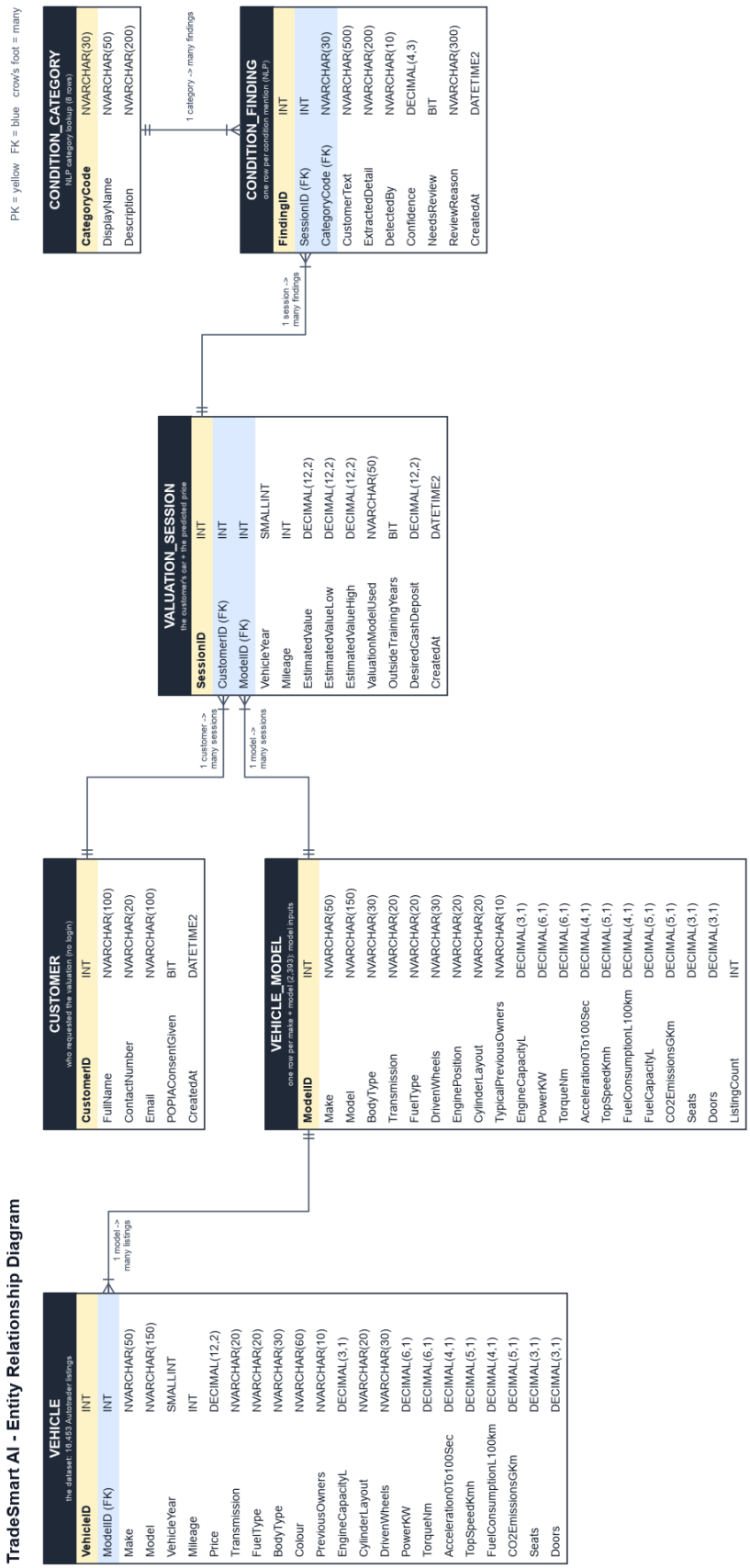


Figure 1: Entity relationship diagram (rotated to fit the page)

8. Model Evaluation and Time Series Analysis

8.1 How accuracy is evaluated

The data was split into **80% for training (13 162 cars) and 20% for testing (3 291 cars)**. The test cars are never used for training or tuning, so they show how the model performs on vehicles it has not seen. On the training data, **10-fold cross-validation** (training on nine tenths and testing on the remaining tenth, ten times) checks that the result is stable rather than lucky. Three measures are reported:

- **MAE (mean absolute error):** the average difference between predicted and actual price, in rand.
- **RMSE (root mean squared error):** like MAE but punishes large mistakes more heavily.
- **R^2 (coefficient of determination):** the share of the variation in prices that the model explains (1.0 is perfect).

8.2 Results

Model	MAE	RMSE	R^2	10-fold CV MAE
Depreciation baseline	R59 981	R163 204	0.9021	-
Linear Regression	R92 169	R181 580	0.8788	R91 281 ± R5 925
Random Forest	R43 196	R107 392	0.9576	R45 683 ± R5 796
XGBoost (tuned, used by the app)	R39 116	R85 415	0.9732	R42 404 ± R4 916
Deep Learning (MLP)	R51 529	R148 098	0.9194	-

The table shows each model on the main test split. Every model except Linear Regression beats the depreciation baseline; Linear Regression cannot capture the fact that cheap and expensive cars lose value very differently, which supports the choice of non-linear models. The features XGBoost relies on most are: Power maximum (detail), Brand = Rolls-Royce, Cylinder layout and quantity = V12, Torque maximum, Engine position = Front, Body Type = Super sports.

How reliable is this result? One split can be lucky or unlucky, so the XGBoost model was retrained and tested on different splits of the data (tradesmart-ml/src/split_experiments.py): four train/test ratios, each with five different random shuffles of the 16 453 cars.

Train / test	Training cars	Test cars	R^2 average (range)	Average error (MAE)	Within ±15%
60 / 40	9 871	6 582	0.918 (0.902-0.947)	R42 652 ± R545	87.1%
70 / 30	11 517	4 936	0.920 (0.892-0.946)	R42 152 ± R906	87.5%
80 / 20	13 162	3 291	0.911 (0.859-0.952)	R41 905 ± R2 130	87.8%
90 / 10	14 807	1 646	0.956 (0.928-0.976)	R40 445 ± R1 478	88.0%

The average error is consistent: about R40 445 to R42 652 in every split, with around 88% of estimates within 15% of the real price. R^2 varies more, from 0.86 to 0.95 across 80/20 shuffles, because it is dominated by a handful of very expensive vehicles (up to R12 million): when one of them lands in the test set and is mispredicted, R^2 drops noticeably. The main split (R^2 0.973) is at the favourable end of that range, so the figure we report, and the one the application shows customers, is the average over five random 80/20 splits: **R^2 0.91, average error R41 905**.

How much training data is needed? The model was also trained on random samples of 1 000 to 13 162 cars, always tested on the same 3 291 unseen cars (Figure 2). Accuracy rises from R^2 0.81 with 1 000 cars to 0.97 with all 13 162, and the average error falls from R71 399 to R39 025. With only 3 000 training cars the model would reach R^2 0.90, which shows why the full training set matters and suggests that more data would improve it further (Section 9.2).

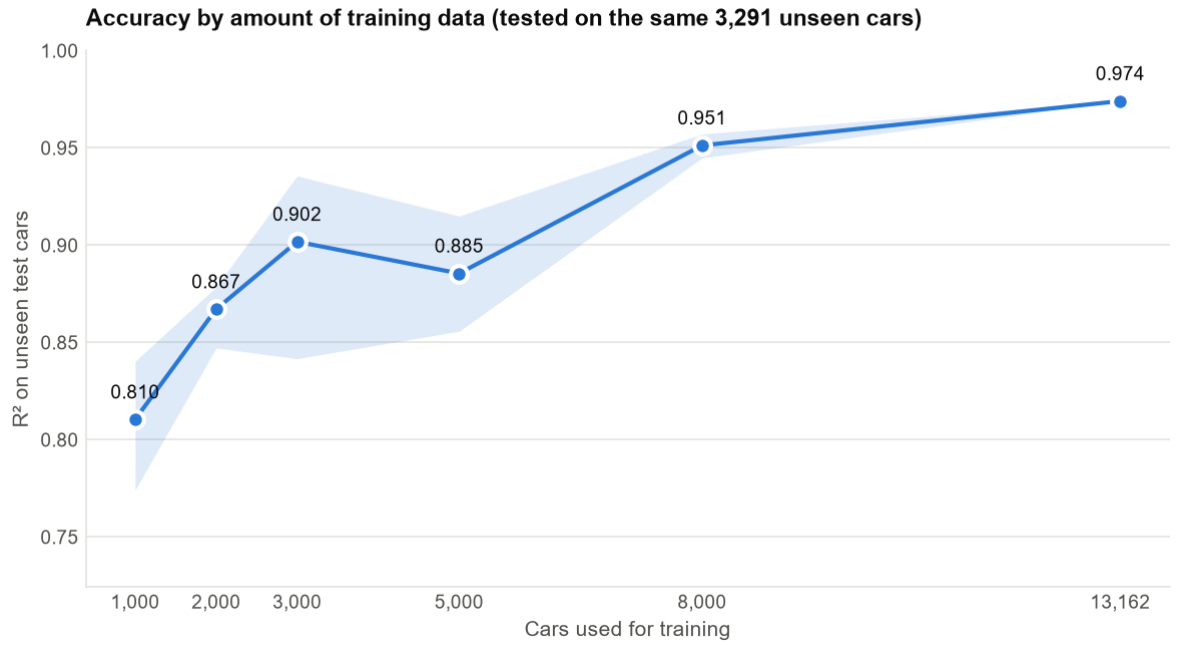


Figure 2: Accuracy by amount of training data

8.3 Time series analysis: how a car's value changes over time

A car's value over time is the core of any trade-in estimate. The dataset is a single snapshot (April 2023) rather than prices recorded month by month, but each listing has a model year, so the same model appears at several ages at once: a 2023, a 2021 and a 2019 Polo Vivo listed side by side. Comparing these gives the value-over-time curve that a price history would show (tradesmart-ml/src/time_series_analysis.py).

Overall averages would be misleading: the median price of all 2021 cars (R349 900) is lower than that of all 2020 cars (R379 750), only because a different mix of models was listed for each year. The analysis therefore compares each model only with itself: for each of 1 289 models listed in more than one year, the median price for each older year is divided by the median price for its newest year. An exponential depreciation curve, value(t) = value(0) × e^(−rt), was then fitted to these 2 499 comparisons.

Years older	Value retained (observed median)	Comparisons	Fitted curve
1	91.7%	1063	90.5%
2	83.7%	676	81.9%
3	75.7%	399	74.1%
4	67.1%	232	67.1%
5	62.5%	129	60.7%

Cars lose about **9.5% of their value per year**, and the fitted curve stays within 1.8% of the observed values at every age (Figure 3).

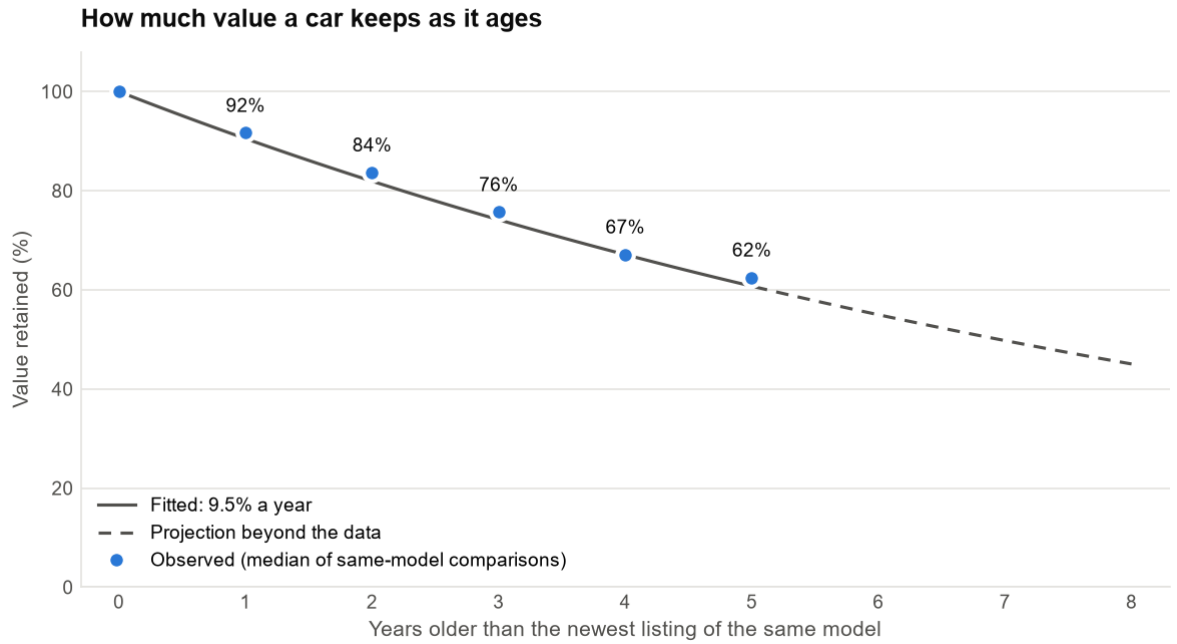


Figure 3: Value retained by age, observed and fitted

Depreciation differs by body type (Figure 4): bakkies hold their value best (extended cab 6.3% a year, double cab 8.2%), while sedans lose value fastest (11.3%). This is useful to the dealership when deciding which trade-ins to accept and how long to keep them in stock.

Body type	Annual depreciation	Comparisons
Extended cab	6.3%	31
Double cab	8.2%	257
Single cab	8.9%	151
Hatchback	8.9%	555
Coupé	9.5%	40
SUV	9.8%	1027
MPV	10.3%	117
Sedan	11.3%	244

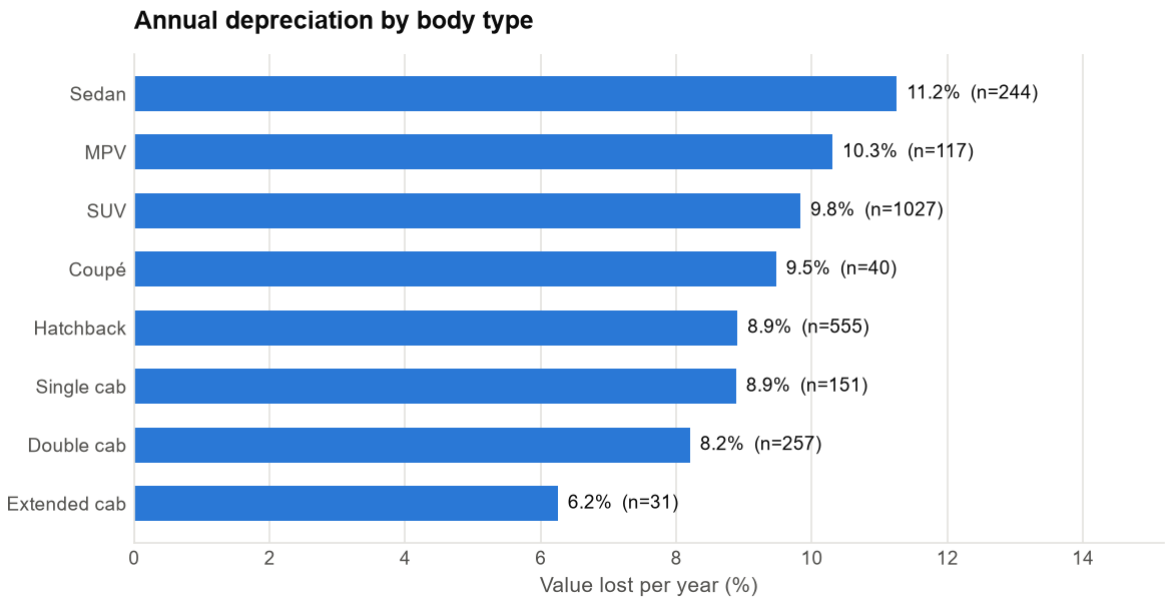


Figure 4: Annual depreciation by body type

The fitted rate can forecast a vehicle's value in future years. For example, a 2019 Volkswagen Polo Vivo Hatch 1.4 Trendline had a median listing price of R169 900 in 2023 (34 listings); Figure 5 projects its value to R103 191 by 2028.

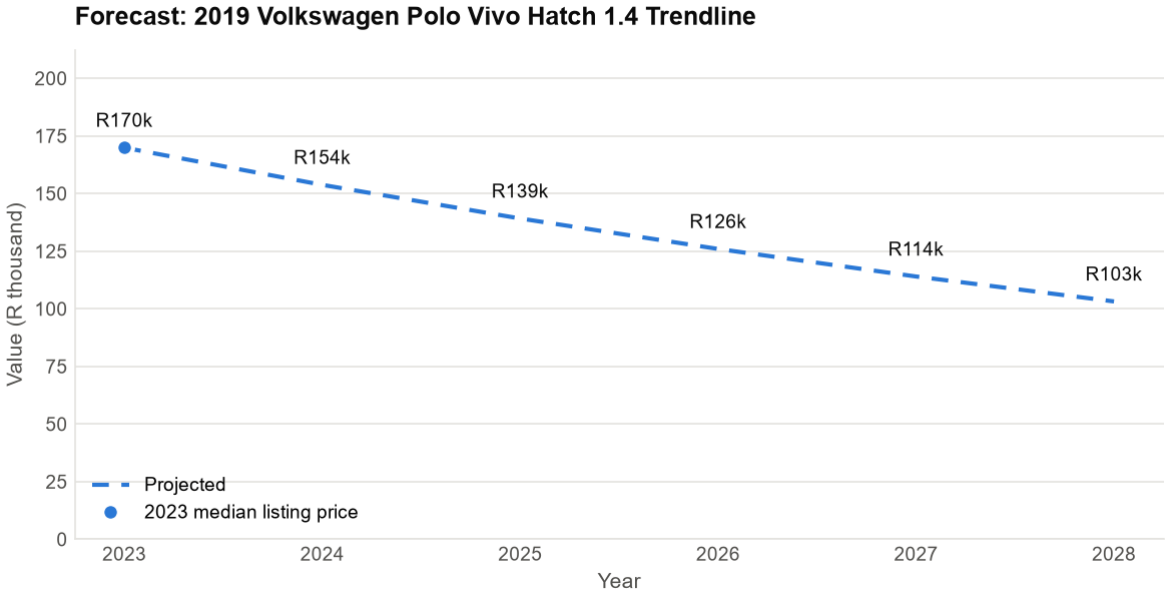


Figure 5: Value forecast for an example vehicle

Limitations. The curve is measured for ages up to five years only, and the forecast assumes depreciation continues at the same rate. Collecting the same listings repeatedly over time would allow a true time series model such as ARIMA or Prophet, which could also detect seasonal effects and market shifts (Section 9.2).

9. Solution Techniques and Improving Accuracy

9.1 Techniques used

- **Feature engineering:** vehicle age is derived from the model year, and a model-popularity feature counts how often each exact model is listed (there are too many distinct model names to use them directly).
- **Preprocessing pipeline:** missing values are filled (median or most common value), numbers are scaled and categories one-hot encoded, fitted on training data only so that no information leaks from the test set.
- **Typical-specification lookup:** fills in the specifications a customer cannot be expected to know (Section 6).
- **Cross-validation and a fixed test set:** for fair comparison of all models.
- **Hyperparameter tuning:** see below.
- **Honest uncertainty:** the estimate is shown as a range based on the measured error, and vehicles outside the 2018-2023 training years are flagged as low-confidence instead of being presented as normal results.

9.2 How the model's accuracy is improved

1. Hyperparameter tuning (done). The first XGBoost model used hand-picked settings. A randomized search tried 40 combinations of settings (number of trees, tree depth, learning rate, sampling rates and regularisation), each evaluated by 5-fold cross-validation on the training data only (Bergstra and Bengio, 2012). The best combination was then tested once on the unseen cars of the main test split:

	MAE	RMSE	R^2
Original settings	R45 607	R87 427	0.9719
Tuned settings (model used by the app)	R39 116	R85 415	0.9732
Change	-14.2%	-2.3%	

The tuned settings are now part of the training script, so the model the application uses is the tuned one.

2. Correcting the data (done). Fixing the three data errors in Section 7.3 gave the model correct fuel-consumption and brand information to learn from.

3. Regular retraining (process in place). Car prices change over time. The documented pipeline (correct the data, train, copy the model into the application) can be rerun on newly collected listings, for example every quarter, so that the model keeps up with the market.

4. Learning from real outcomes (next step). Each valuation session is stored in the database. Adding the price actually agreed with each customer would allow the model to be calibrated from advertised prices to real trade-in values, and to be measured against real outcomes rather than listings.

5. Using condition data (future). Once enough sessions have both NLP condition findings and agreed prices, the findings can become model features, so that condition affects the estimate based on evidence rather than guesswork.

6. Price history and time series models (future). Collecting listings repeatedly over time would turn the snapshot into a true time series (Section 8.3).

7. Growing the NLP training data (future). Real, anonymised customer answers can be labelled and added to the NLP classifier's training examples.

10. Natural Language Processing

The chatbot asks whether the car has been in an accident and whether it has dents, windscreen damage, respray work or aftermarket modifications. Customers answer in their own words. NLP turns those answers into structured findings in four steps (tradesmart-flask/condition_nlp.py):

1. **Splitting:** the answer is split into separate mentions ("dent on the bonnet and a chip in the windscreen" becomes two), and negative mentions such as "windscreen is fine" are ignored.
2. **Intent classification:** each mention is classified into one of eight categories by a model trained with TF-IDF text features and Logistic Regression on 102 labelled example answers. It scored 96% accuracy on 26 held-out examples. The examples were written by the group because no real customer answers exist yet; this small test set makes the accuracy indicative only.
3. **Entity extraction:** rules extract useful details, such as dent size ("about the size of a golf ball" is small, "4 cm" is medium), the type of modification, and whether it was fitted by a reputable installer or by the owner.
4. **Consistency checks:** a finding is flagged for review when the model's confidence is low, the answer is vague ("not sure"), or the answer contradicts itself, for example "no accidents" followed by "resprayed after a crash".

Examples from testing the application:

Customer wrote	Category	Detail extracted	Confidence / flag
small dent about the size of a golf ball on the door	Dents / scratches	small, about the size of a golf ball	87%
aftermarket rims fitted by a professional shop	Aftermarket modifications	wheels; reputable installer	82%
bumper resprayed after a crash (customer said: no accidents)	Respray / paintwork	-	40%, flagged: contradiction
windscreen is fine	(ignored: negative mention)	-	-

The findings are shown to the customer and stored in the CONDITION_FINDING table, but they **do not change the price**: the dataset contains no information about condition, so there is no evidence of how much a dent should reduce the value. Using condition in the price is part of the improvement plan (Section 9.2). Speech recognition was not implemented; letting customers speak their answers would be a natural extension, as the same NLP steps could process the transcribed text.

11. Deep Learning

A **multi-layer perceptron (MLP)**, a feed-forward deep neural network, was trained with TensorFlow/Keras on exactly the same features and data split as the other models (tradesmart-ml/src/train_deep_learning.py). Its architecture is: Dense(128)-BN-Dropout(0.2)-Dense(64)-BN-Dropout(0.2)-Dense(32)-Dense(1), Adam lr=1e-3, target standardised.

- **Target scaling:** prices range from about R59 000 to R12 million, so they are standardised before training to keep the learning stable.
- **Batch normalisation and dropout** reduce overfitting to the roughly 13 000 training cars.
- **Early stopping and learning-rate reduction:** training stops when the error on a validation split stops improving (it stopped after 134 epochs) and keeps the best weights.

The MLP reached R^2 0.919 (MAE R51 529). It clearly beats Linear Regression but not the tree ensembles. This is a common result for structured, tabular data of this size, where gradient-boosted trees usually outperform neural networks; deep learning tends to lead on large volumes of unstructured data. That is where it fits a future version of TradeSmart AI: a **convolutional neural network (CNN)** could assess damage from photos of the car, and a **speech-recognition** model could let customers speak their answers.

12. Chatbot

The customer-facing part of TradeSmart AI is a guided chatbot on the dealership's website. It replaces the first conversation with a sales consultant and works at any time, without a login. The conversation has five steps:

1. **Vehicle:** the customer starts typing a make and model, and matching vehicles from the 2 393 known models appear as they type.
2. **Year and mileage:** checked immediately; common formats such as "85 000", "85,000" and "85k" are understood, and unreadable values get a clear request to try again.
3. **Condition:** the customer describes the car in their own words or skips; the bot replies with what the NLP model noted.
4. **Deposit:** an optional cash deposit towards another vehicle.
5. **Contact details:** the customer's name, plus phone and email only if they tick the POPIA consent box.

The bot then shows the estimate, its range, the model's measured accuracy and any low-confidence warning, and saves the session to the database. Every submission is also appended to a log file that opens in Excel, giving the dealership a simple record of every enquiry without customers needing an account. The chatbot is built with Flask (Python) and htmx, so all logic runs on the server in Python.

13. Practical Solution

The working solution consists of three parts, each in its own folder:

- **tradesmart-flask:** the web application and chatbot; started with a one-click launcher that installs everything it needs on first run.
- **tradesmart-ml:** data correction, training and evaluation of all models, the NLP classifier and the time series analysis.
- **database:** the SQL Server script that creates the six tables and loads their data (including all 16 453 listings), and the ERD.

References

Adewoyin, V. (2023) used-car-price-prediction [Source code and dataset]. GitHub. Available at: https://github.com/vaadewoyin/used-car-price-prediction (MIT licence).

AutoTrader South Africa (2023) Used cars for sale. Available at: https://www.autotrader.co.za (data collected 23-24 April 2023 by Adewoyin, 2023).

Bergstra, J. and Bengio, Y. (2012) 'Random search for hyper-parameter optimization', Journal of Machine Learning Research, 13, pp. 281-305.

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5-32.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794.

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825-2830.

Abadi, M. et al. (2016) 'TensorFlow: A system for large-scale machine learning', 12th USENIX Symposium on Operating Systems Design and Implementation, pp. 265-283.

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.